

Finite-State Transducers in OCamlFLAT/OFLAT

João Santos

NOVA University Lisbon

July 2026

Overview

- Context, motivation and objectives
- What are FSTs
- OCamlFLAT/OFLAT system overview
- Implementation decisions/challenges
- Future work/Conclusion

Finite Automata to Transducers

Concept

- **Finite Automata (FA) as Recognizers:** Process an input string to yield a binary decision (Accepted / Rejected).
- **Finite-State Transducers (FST) as Transformers:** Process an input string and actively generate a corresponding output sequence.

Context and Objectives

Context

- **Theory of Computation** is a foundational area in computer science.
- Formal Languages and Automata Theory (FLAT) is key to modeling computation and designing compilers.
- Educational tools like **OCamlFLAT/OFLAT** help visualize and simulate formal models.
- The system supports many models, but **lacks support for Finite-State Transducers (FSTs)**.

Objectives

- Extend OCamlFLAT/OFLAT with full support for FSTs.
- Implement core operations: translation, recognition, determinization, minimization.
- Design interactive FST support in the web interface (OFLAT).
- Ensure consistency with existing models and support exercises.

Motivation

Why

- Formal Languages and Automata Theory are fundamental in CS education.
- Students struggle with abstract concepts.
- Visual and interactive tools help to understand these concepts.
- FSTs are underrepresented in pedagogical tools.
- OCamlFLAT/OFLAT lacks FST support.

What are Finite-State Transducers?

- FSTs are understood as an extension of finite automata by producing outputs during computation.
- They are defined over an input alphabet Σ and output alphabet Γ .
- **Common operations:** union, concatenation, Kleene star, composition, projection.

Finite-State Transducer Model

Formal

Definition:

- Formally, an FST is defined as a 6-tuple $(Q, \Sigma, \Gamma, I, F, \delta)$, where:
 - Q is a finite set, the set of **states**;
 - Σ is a finite set, called the **input alphabet**;
 - Γ is a finite set, called the **output alphabet**;
 - $I \subseteq Q$ is the set of **initial states**;
 - $F \subseteq Q$ is the set of **final states**;
 - $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \times Q$ is the **transition relation**, where ε represents the empty string.
- Translates input strings into output strings.

Finite-State Transducer Model

Model

- **Alphabets:** Defined by distinct Input (Σ) and Output (Γ) alphabets.
- **State Configuration:** Consists of a finite set of states, an initial state, and designated accepting (final) states.
- **Transitions:** Edges evaluate an input symbol and emit an output symbol (e.g., $a : x$).
- **Acceptance:** A valid execution must fully consume the input string, reach an accepting state, and reliably produce the expected output.

Mealy and Moore Machines

Special Cases

- **Mealy Machines:** Output is determined by both the current state and the current input symbol (outputs are bound to transitions).
- **Moore Machines:** Output is determined exclusively by the current state (outputs are bound to the states themselves).
- **OFLAT Integration:** The system dynamically analyzes user-created FSTs to identify and classify these specific properties in real-time.

OCamlFLAT/OFLAT System Overview

- **OCamlFLAT Library:**

- Logic for formal models
- Modular and extensible architecture based on OCaml.

- **OFLAT Web Application:**

- Pedagogical and interactive browser interface.
- Allows creation, editing and simulation of models.
- Pedagogical exercises for training and self evaluation
- Real-time visual feedback for word recognition and transducing.
- The implementation follows the Model-View-Controller (MVC) pattern.

- **Pedagogical Goals:**

- Help students visualize abstract computation steps.
- Enable experimentation and learning through interaction.

OCamIFLAT/OFLAT System Overview

OFLAT
Version 5.0

New model

Edit model

Fit to box

Generate words with size X

Test word

Trace

Step-by-step word acceptance

◀

Start

▶

Auto

Convert to

Import model

Export model

Save model

Delete models

Predefined examples

- Finite State Transducers -

fst_1

fst_ND

fst_ND2

fst_complex_ND

fst_det_min

fst_M

fst_Moore

fst_Mealy

About

Feedback

Settings

X

Action

Help

See model specification

Clean

Deterministic

Minimize automaton

Productive

Accessible

Useful

Clean Automaton

Refresh layout

Show Table View

Model: Finite State Transducer

Name: fst_det_min

N° States: 3

N° Transitions: 7

Deterministic: False

Mealy: False

Moore: False

The automaton has no useless states.

Developed in NOVA-LINCS by the projects [Factor](#) and [LEAEs](#) / Co-financed by [Tosco Foundation](#) and [INRIA Foundation](#)

Technologies Behind OFLAT

- **js_of_ocaml:**

- Interoperability solution.
- Compiles OCaml bytecode to JavaScript.
- Supports bindings to native JS libraries (e.g., DOM, Cytoscape.js).

- **Cytoscape.js for Visualization:**

- Used to draw graphs representing automata and trees.
- Allows dynamic interaction: add/edit states, transitions, layout.

- **Web Foundations:**

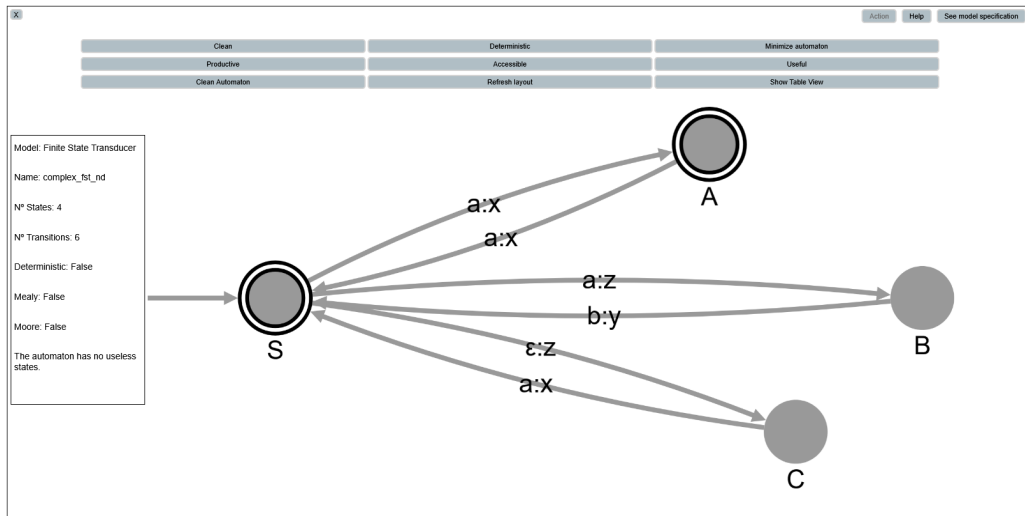
- HTML for structure, JavaScript for interactivity, DOM for manipulation.
- OFLAT dynamically updates based on user actions.

User Interface Integration

UI

- **Seamless Access:** "Finite-State Transducer" is now natively available in the OFLAT "New Model" dropdown menu.
- **Intuitive Editing:** Users can visually build FSTs via the drag-and-drop workspace.
- **Transition Syntax:** Introduces a clean input,output prompt format, which renders as standard input:output labels on the graph edges.
- **Epsilon Support:** Empty-symbol settings (ϵ , λ , $\sim$) are fully supported for both input reading and output generation.

User Interface Integration



Exercise Support & Checking

Checking

- **FA Validation:** FA exercises only verify if words are accepted or rejected.
- **FST Validation:** FST exercises validate the inputs and outputs together so it only accepts the input word if it generates the expected output. Implemented a new, output-sensitive exercise validator.

Exercise Support & Checking

X Action Help See model specification

Clean	Deterministic	Minimize automaton
Productive	Accessible	Useful
Clean Automaton	Refresh layout	Show Table View

Model: Finite State Transducer

Name: fst_Mealy

Nº States: 1

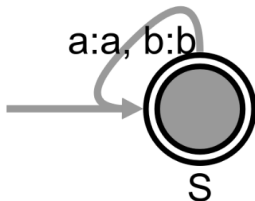
Nº Transitions: 2

Deterministic: True

Mealy: True

Moore: False

The automaton has no useless states.



Exercise

Problem: Create a deterministic MEALY MACHINE (Finite State Transducer) over input alphabet {a, b} that swaps every symbol: outputs 'b' for every 'a' read and 'a' for every 'b' read.

Wrong answer

Properties to satisfy:

transducer
deterministic
mealy

Words to accept:

'a->b'
'b->a'
'ab->ba'
'ba->ab'
'aab->bba'

Words to reject:

Verify

Implementation Challenges

Risks

- **Data Structure changes:** Adapting core OCaml types to support 4-tuple transitions (source, input, output, destination) instead of standard 3-tuples.
- **Algorithmic Overhaul:** Classic operations (like Determinization and Minimization) are more complex, requiring logic to handle output conflicts as well as other specific cases
- **UI Consistency:** Adding new panels and behaviors without disrupting the legacy OFLAT design language.
- **Evaluation Criteria:** Redesigning the exercise checking engine to evaluate string generation rather than just boolean acceptance.

- **Further models:** Explore other transducer models (e.g., weighted, probabilistic) to expand the didactic scope while reusing the existing interaction patterns.
- **Enhanced Feedback:** Provide more granular, visual error reporting during FST determinization failures (e.g., highlighting conflicting output paths).
- **Expanded Curriculum:** Develop a comprehensive suite of default FST exercises for educators and students.
- **Robust Testing:** Increase automated test coverage for complex transducer edge cases.

Conclusion

Wrap-Up

- **Successful Integration:** OFLAT now features robust, interactive support for Finite-State Transducers.
- **Enhanced Learning:** Students can intuitively create, visualize, manipulate, and test transducers on the web.
- **Extra work:** Besides the proposed work, there were implemented various general bug fixes and small features like extending the help pages.